

Ensemble Learning

From weak models to strong predictors

AdaBoost, Gradient Boosting, Bagging, and Random Forests

COMP 5212: Machine Learning

Zihan Zhang

From one score to a sum of rules

Linear and logistic regression start from one score: $F(x) = \beta_0 + x^\top \beta$.

Regression

Prediction: $\hat{y} = F(x)$.

$$\ell(y, F) = \frac{1}{2}(y - F)^2.$$

Binary classification

Prediction: $\hat{p} = s(F(x))$, $y \in \{0, 1\}$.

$$\ell(y, F) = \log(1 + e^F) - yF.$$

From one score to a sum of rules

Linear and logistic regression start from one score: $F(x) = \beta_0 + x^\top \beta$.

Regression

Prediction: $\hat{y} = F(x)$.

$$\ell(y, F) = \frac{1}{2}(y - F)^2.$$

Binary classification

Prediction: $\hat{p} = s(F(x))$, $y \in \{0, 1\}$.

$$\ell(y, F) = \log(1 + e^F) - yF.$$

Learn the score as a sum of fitted rules:

$$F_T(x) = F_0(x) + \sum_{t=1}^T \alpha_t h_t(x).$$

Each rule can be simple; their combination can express a richer function.

Learning Goals

By the end of the lecture, you should be able to:

- explain how AdaBoost combines simple rules into a stronger classifier;
- derive its weights and state what its training-error bound guarantees;
- connect residual fitting to gradient descent in prediction space;
- explain how bagging and random forests stabilize tree predictions.

AdaBoost: observe $\rightarrow$ derive Gradient boosting: correct the prediction
Then: a brief tree primer $\rightarrow$ bagging and random forests

Where do ensembles become useful?

Four documented applications accompany the algorithms

Predict a movie rating

User/movie history $\rightarrow$ rating

Blend recommendation models

Estimate an ad's click probability

User/ad context $\rightarrow$ probability

Boosted trees + logistic regression

Forecast electricity demand

Weather and history $\rightarrow$ load

Gradient boosting with smooth functions

Track a player's body

Depth pixels $\rightarrow$ body parts

Randomized decision forests

What is predicted? What is combined? How is success measured?

Phase 1: AdaBoost

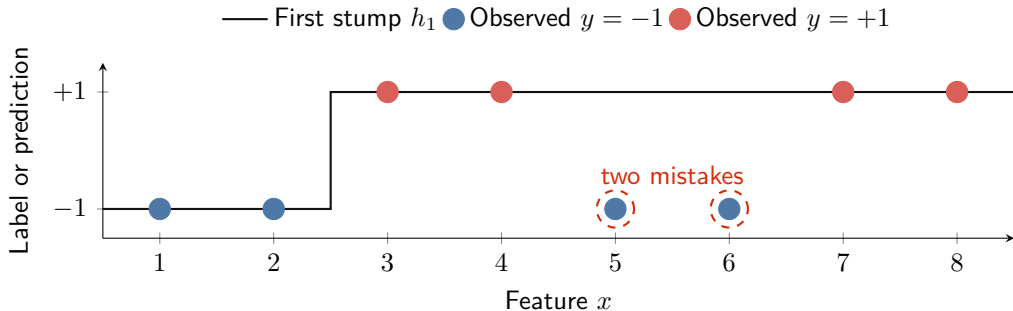
We can replace one score with a sum of simple rules.

How should simple rules
learn to work together?

Next: Watch AdaBoost change the observation weights, then combine the rules by a weighted vote.

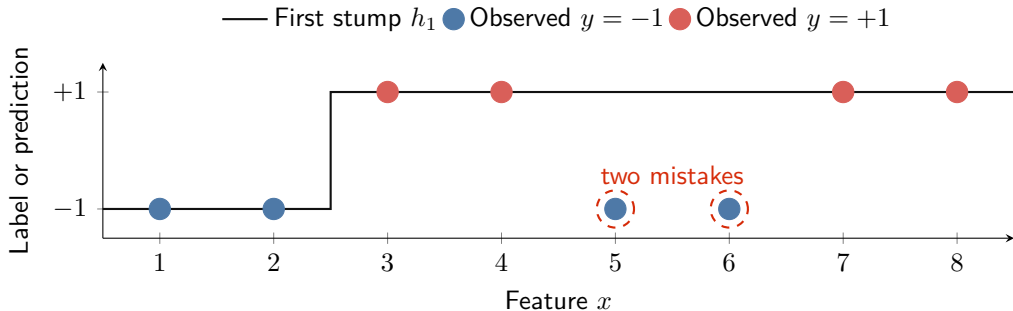
Can Weak Rules Work Together?

Eight constructed observations. A threshold rule (a *decision stump*) makes one split.



Can Weak Rules Work Together?

Eight constructed observations. A threshold rule (a *decision stump*) makes one split.



Keep the first rule. Which observations should the next rule emphasize?

Two Kinds of Weight

Use labels $y_i \in \{-1, +1\}$ and rules $h_t(x) \in \{-1, +1\}$.

Weights on observations: $D_t(i)$

$$D_t(i) \geq 0, \quad \sum_i D_t(i) = 1.$$

Weighted error of the next rule:

$$\epsilon_t = \sum_i D_t(i) \mathbf{1}\{y_i \neq h_t(x_i)\}.$$

Start with $D_1(i) = 1/n$; change the emphasis after each fitted rule.

Weights on rules: α_t

$$F_T(x) = \sum_{t=1}^T \alpha_t h_t(x).$$

Prediction is a weighted vote:

$$\hat{y}(x) = \text{sign}(F_T(x)).$$

A larger positive α_t gives that rule a stronger vote.

Fit a rule, set its vote weight, then change which observations matter most.

AdaBoost: The Algorithm

Initialize $F_0(x) = 0$ and $D_1(i) = 1/n$. For $t = 1, \dots, T$:

1. Fit h_t using observation weights D_t ; calculate

$$\epsilon_t = \sum_i D_t(i) \mathbf{1}\{y_i \neq h_t(x_i)\}.$$

2. If $0 < \epsilon_t < 1/2$, set $\alpha_t = \frac{1}{2} \log \frac{1 - \epsilon_t}{\epsilon_t}$.
3. Update the score: $F_t(x) = F_{t-1}(x) + \alpha_t h_t(x)$.
4. Update and normalize: $D_{t+1}(i) \propto D_t(i) e^{-\alpha_t y_i h_t(x_i)}$.

Predict $\hat{y}(x) = \text{sign}(F_T(x))$; fix a rule for ties.

Stop for a perfect learner; stop or revise the rule class if no rule has $\epsilon_t < 1/2$.

One Update, New Weights

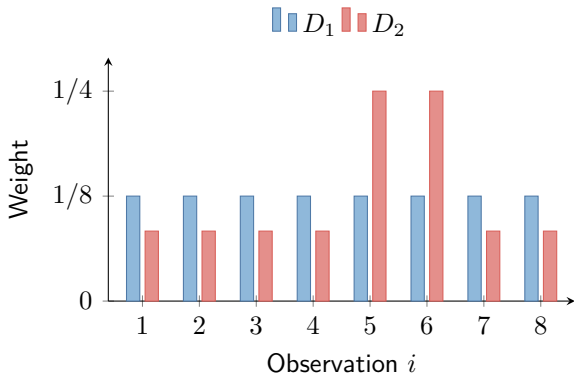
The first stump misses points 5 and 6.

$$\epsilon_1 = \frac{2}{8} = \frac{1}{4}, \quad \alpha_1 = \frac{1}{2} \log 3 \approx 0.549.$$

After normalization:

$$D_2(i) = \begin{cases} 1/4, & i \in \{5, 6\}, \\ 1/12, & \text{otherwise.} \end{cases}$$

The two mistakes now receive 1/2 of the total weight.



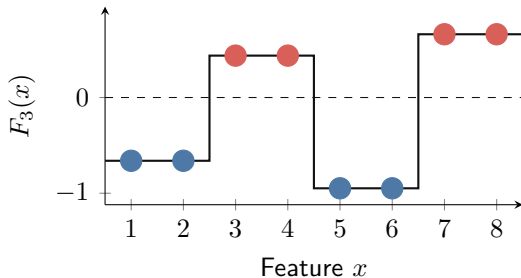
Three Stumps, One Classifier

t	$h_t(x) = +1$ when	ϵ_t	α_t
1	$x \geq 2.5$	$1/4$	0.549
2	$x \geq 6.5$	$1/6$	0.805
3	$x < 4.5$	$1/5$	0.693

$$F_3 = 0.549h_1 + 0.805h_2 + 0.693h_3.$$

$$F_3(5) = 0.549 - 0.805 - 0.693 < 0,$$

$$F_3(3) = 0.549 - 0.805 + 0.693 > 0.$$



Blue: $y = -1$. Red: $y = +1$.

Training error after rounds 1, 2, 3: 25%, 25%, 0%. No single stump can fit this pattern.

Phase 2: Why AdaBoost Works

Three fitted stumps separated a pattern no single stump could fit.

Why these observation weights
and vote weights?

Next: Choose a loss. Derive one update. Then bound the ensemble's training error.

A Loss for the Accumulated Vote

Labels: $y_i, h_t(x) \in \{-1, +1\}$.

$$F_T(x) = \sum_{t=1}^T \alpha_t h_t(x).$$

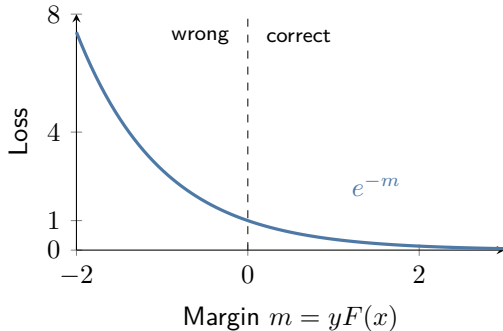
Predict the sign of $F_T(x)$.

Margin: $m_i = y_i F_T(x_i)$.

$m_i > 0$: correct. $m_i < 0$: incorrect.

Fit by reducing exponential loss:

$$J(F) = \frac{1}{n} \sum_{i=1}^n e^{-y_i F(x_i)}.$$



Stagewise Fitting and Weights

Hold F_{t-1} fixed and add one new weak learner: $F_t = F_{t-1} + \alpha h$.

$$J(F_{t-1} + \alpha h) = \frac{1}{n} \sum_i e^{-y_i F_{t-1}(x_i)} e^{-\alpha y_i h(x_i)}.$$

Stagewise Fitting and Weights

Hold F_{t-1} fixed and add one new weak learner: $F_t = F_{t-1} + \alpha h$.

$$J(F_{t-1} + \alpha h) = \frac{1}{n} \sum_i e^{-y_i F_{t-1}(x_i)} e^{-\alpha y_i h(x_i)}.$$

Normalize the old losses into a distribution over observations:

$$D_t(i) = \frac{e^{-y_i F_{t-1}(x_i)}}{\sum_j e^{-y_j F_{t-1}(x_j)}}, \quad \sum_i D_t(i) = 1.$$

$$J(F_{t-1} + \alpha h) = J(F_{t-1}) \underbrace{\sum_i D_t(i) e^{-\alpha y_i h(x_i)}}_{Z_t(\alpha, h)}.$$

Choosing a Weak Learner

For a binary prediction, $y_i h(x_i)$ is either $+1$ or -1 .

$$\epsilon_t(h) = \sum_i D_t(i) \mathbf{1}\{y_i \neq h(x_i)\}.$$

$$\begin{aligned} Z_t(\alpha, h) &= \sum_{\text{correct}} D_t(i) e^{-\alpha} + \sum_{\text{incorrect}} D_t(i) e^{\alpha} \\ &= (1 - \epsilon_t(h)) e^{-\alpha} + \epsilon_t(h) e^{\alpha} \\ &= e^{-\alpha} + \epsilon_t(h) (e^{\alpha} - e^{-\alpha}). \end{aligned}$$

Choosing a Weak Learner

For a binary prediction, $y_i h(x_i)$ is either $+1$ or -1 .

$$\epsilon_t(h) = \sum_i D_t(i) \mathbf{1}\{y_i \neq h(x_i)\}.$$

$$\begin{aligned} Z_t(\alpha, h) &= \sum_{\text{correct}} D_t(i) e^{-\alpha} + \sum_{\text{incorrect}} D_t(i) e^{\alpha} \\ &= (1 - \epsilon_t(h)) e^{-\alpha} + \epsilon_t(h) e^{\alpha} \\ &= e^{-\alpha} + \epsilon_t(h) (e^{\alpha} - e^{-\alpha}). \end{aligned}$$

For $\alpha > 0$, choose a weak learner with the smallest weighted error:

$$h_t \in \arg \min_{h \in \mathcal{H}} \epsilon_t(h).$$

Deriving the Vote Weight

Fix h_t and write $\epsilon_t = \epsilon_t(h_t)$, with $0 < \epsilon_t < \frac{1}{2}$.

$$Z_t(\alpha) = (1 - \epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha}.$$

$$\frac{dZ_t}{d\alpha} = -(1 - \epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha} = 0 \quad \implies \quad e^{2\alpha} = \frac{1 - \epsilon_t}{\epsilon_t}.$$

Deriving the Vote Weight

Fix h_t and write $\epsilon_t = \epsilon_t(h_t)$, with $0 < \epsilon_t < \frac{1}{2}$.

$$Z_t(\alpha) = (1 - \epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha}.$$

$$\frac{dZ_t}{d\alpha} = -(1 - \epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha} = 0 \quad \implies \quad e^{2\alpha} = \frac{1 - \epsilon_t}{\epsilon_t}.$$

$$\boxed{\alpha_t = \frac{1}{2} \log \frac{1 - \epsilon_t}{\epsilon_t}} \quad \text{and} \quad Z_t''(\alpha) > 0.$$

Smaller weighted error gives a larger vote. $\epsilon_t \uparrow \frac{1}{2} \implies \alpha_t \downarrow 0$.

Why Mistakes Receive More Weight

Insert $F_t = F_{t-1} + \alpha_t h_t$ into the definition of D_{t+1} .

$$D_{t+1}(i) = \frac{D_t(i)e^{-\alpha_t y_i h_t(x_i)}}{Z_t}, \quad Z_t = \sum_j D_t(j)e^{-\alpha_t y_j h_t(x_j)}.$$

$$e^{-\alpha_t y_i h_t(x_i)} = \begin{cases} e^{-\alpha_t}, & \text{correct prediction,} \\ e^{+\alpha_t}, & \text{incorrect prediction.} \end{cases}$$

Why Mistakes Receive More Weight

Insert $F_t = F_{t-1} + \alpha_t h_t$ into the definition of D_{t+1} .

$$D_{t+1}(i) = \frac{D_t(i)e^{-\alpha_t y_i h_t(x_i)}}{Z_t}, \quad Z_t = \sum_j D_t(j)e^{-\alpha_t y_j h_t(x_j)}.$$

$$e^{-\alpha_t y_i h_t(x_i)} = \begin{cases} e^{-\alpha_t}, & \text{correct prediction,} \\ e^{+\alpha_t}, & \text{incorrect prediction.} \end{cases}$$

The weight of a mistake *relative to a correct example* is multiplied by

$$e^{2\alpha_t} = \frac{1 - \epsilon_t}{\epsilon_t} > 1.$$

Bounding the Training Error

Start from $F_0 = 0$, so $J(F_0) = 1$. Count a zero margin as an error.

$$\frac{1}{n} \sum_i \mathbf{1}\{y_i F_T(x_i) \leq 0\} \leq \frac{1}{n} \sum_i e^{-y_i F_T(x_i)} = \prod_{t=1}^T Z_t.$$

At the derived step size, $Z_t = 2\sqrt{\epsilon_t(1 - \epsilon_t)}$.

Bounding the Training Error

Start from $F_0 = 0$, so $J(F_0) = 1$. Count a zero margin as an error.

$$\frac{1}{n} \sum_i \mathbf{1}\{y_i F_T(x_i) \leq 0\} \leq \frac{1}{n} \sum_i e^{-y_i F_T(x_i)} = \prod_{t=1}^T Z_t.$$

At the derived step size, $Z_t = 2\sqrt{\epsilon_t(1 - \epsilon_t)}$.

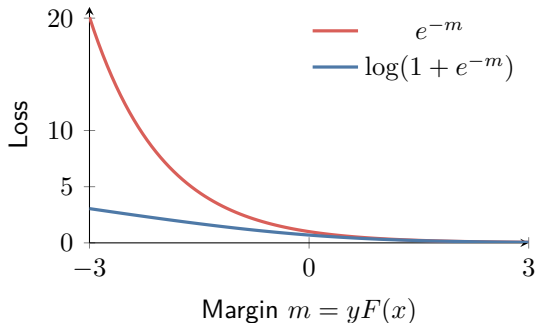
Writing the weak learner's edge as $\delta_t = \frac{1}{2} - \epsilon_t > 0$,

$$\prod_t Z_t = \prod_t \sqrt{1 - 4\delta_t^2} \leq \exp\left(-2 \sum_t \delta_t^2\right).$$

If every round has $\delta_t \geq \delta > 0$, the bound is at most $e^{-2T\delta^2}$.

This controls training error. Generalization still requires separate analysis.

When Labels Are Wrong



Very negative margins produce large losses and large weights.

- A useful signal can receive more attention.
- A mislabeled observation can receive more attention too.
- Select the number of rounds using validation data.

Changing the loss changes which errors dominate the next update.

Back to Logistic Regression

At a fixed x , let $p = \Pr(Y = +1 \mid x)$, with $0 < p < 1$.

$$R(F \mid x) = \mathbb{E}[e^{-YF} \mid x] = pe^{-F} + (1 - p)e^F.$$

$$\frac{\partial R}{\partial F} = -pe^{-F} + (1 - p)e^F = 0 \quad \implies \quad F^*(x) = \frac{1}{2} \log \frac{p}{1 - p}.$$

Back to Logistic Regression

At a fixed x , let $p = \Pr(Y = +1 \mid x)$, with $0 < p < 1$.

$$R(F \mid x) = \mathbb{E}[e^{-YF} \mid x] = pe^{-F} + (1 - p)e^F.$$

$$\frac{\partial R}{\partial F} = -pe^{-F} + (1 - p)e^F = 0 \quad \implies \quad F^*(x) = \frac{1}{2} \log \frac{p}{1 - p}.$$

The unrestricted population optimum satisfies

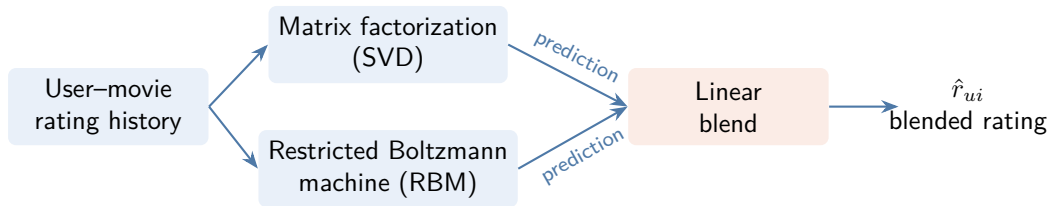
$$p = s(2F^*(x)).$$

AdaBoost optimizes exponential loss.

We can build an additive model by optimizing other losses too.

Movie ratings: two views of taste

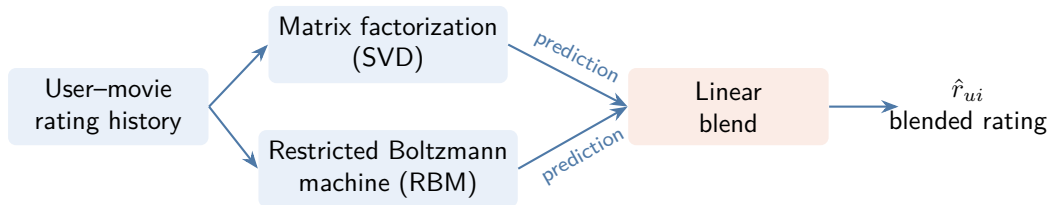
Netflix: predict a user's rating for a movie they have not rated.



Amatriain and Basilico, Netflix Technology Blog (2012). [Source](#)

Movie ratings: two views of taste

Netflix: predict a user's rating for a movie they have not rated.



Prediction blending: a generic linear form

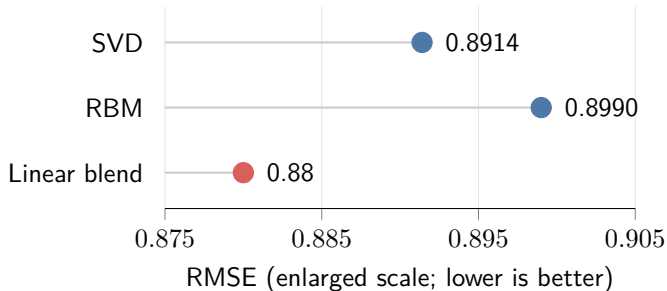
$$\hat{r}_{ui} = b + \beta_1 f_{\text{SVD}}(u, i) + \beta_2 f_{\text{RBM}}(u, i).$$

Combine predictions from models that learn different patterns.

Amatriain and Basilico, Netflix Technology Blog (2012). [Source](#)

A blend improves rating prediction

Netflix's published comparison: offline error in predicted movie ratings.



Lower RMSE is better
The blend improves on both individual predictors.

Netflix reported deploying adapted versions of both algorithms.

Historical account: 2012.

Rating accuracy is one component of recommendation quality.

Amatriain and Basilico (2012), "The Netflix Prize and the Recommendation Problem." [Source](#)

Phase 3: Learning a Correction

We have built an additive classifier and seen how combinations can improve predictions.

What should the next model learn?

Next: Start with the current residuals, then let the loss define the correction targets.

Keep the model; learn a correction

AdaBoost already builds an additive model, one simple function at a time

$$F_m(x) = \underbrace{F_{m-1}(x)}_{\text{keep the current fit}} + \underbrace{\eta_m h_m(x)}_{\text{add a correction}} .$$

For regression, the current prediction may be too high or too low.

Keep the model; learn a correction

AdaBoost already builds an additive model, one simple function at a time

$$F_m(x) = \underbrace{F_{m-1}(x)}_{\text{keep the current fit}} + \underbrace{\eta_m h_m(x)}_{\text{add a correction}} .$$

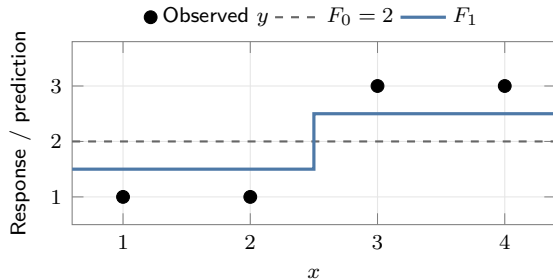
For regression, the current prediction may be too high or too low.

$$\underbrace{y_i - F_{m-1}(x_i)}_{\text{desired correction at } x_i} \longrightarrow \underbrace{h_m(x)}_{\text{a simple fitted function}} .$$

Fit what is left to explain. Then recompute what is left.

One correction lowers squared error

Squared loss: $\ell(y, F) = \frac{1}{2}(y - F)^2$. Start from the mean response



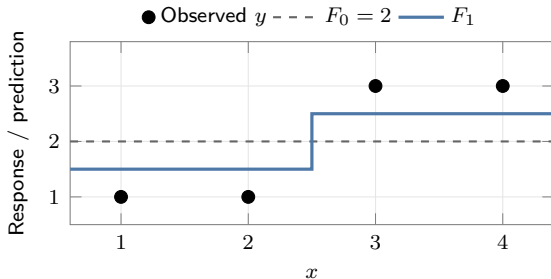
$$y = (1, 1, 3, 3), \quad F_0 = 2.$$

$$r_1 = y - F_0 = (-1, -1, 1, 1).$$

A one-split rule predicts -1 for $x \leq 2.5$ and $+1$ otherwise.

One correction lowers squared error

Squared loss: $\ell(y, F) = \frac{1}{2}(y - F)^2$. Start from the mean response



$$y = (1, 1, 3, 3), \quad F_0 = 2.$$

$$r_1 = y - F_0 = (-1, -1, 1, 1).$$

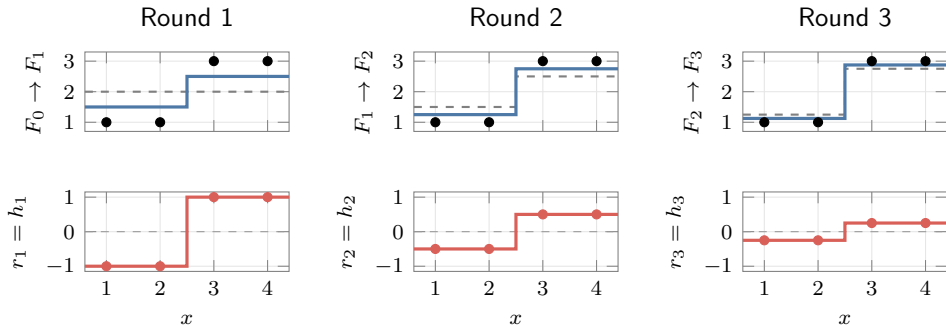
A one-split rule predicts -1 for $x \leq 2.5$ and $+1$ otherwise.

With shrinkage $\nu = \frac{1}{2}$,

$$\begin{aligned} F_1 &= F_0 + \nu h_1 \\ &= (1.5, 1.5, 2.5, 2.5). \end{aligned}$$

Correct what the previous model missed

Same four observations; refit to the new residuals each round; $\nu = \frac{1}{2}$



$$\text{MSE}(F_0) = 1 \quad \longrightarrow \quad \text{MSE}(F_1) = \frac{1}{4} \quad \longrightarrow \quad \text{MSE}(F_2) = \frac{1}{16} \quad \longrightarrow \quad \text{MSE}(F_3) = \frac{1}{64}.$$

Why squared loss leads to residuals

Let $L(F) = \frac{1}{2} \sum_i (y_i - F(x_i))^2$ and $r_i = y_i - F(x_i)$.

Why squared loss leads to residuals

Let $L(F) = \frac{1}{2} \sum_i (y_i - F(x_i))^2$ and $r_i = y_i - F(x_i)$.

$$-\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} = y_i - F(x_i) = r_i.$$

Why squared loss leads to residuals

Let $L(F) = \frac{1}{2} \sum_i (y_i - F(x_i))^2$ and $r_i = y_i - F(x_i)$.

$$-\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} = y_i - F(x_i) = r_i.$$

For a proposed correction $h_i = h(x_i)$,

$$L(F + \eta h) - L(F) = -\eta \underbrace{\sum_i r_i h_i}_{\text{alignment with the residual}} + \frac{\eta^2}{2} \sum_i h_i^2.$$

If $\sum_i r_i h_i > 0$, a sufficiently small positive step lowers the training loss.

Gradient descent in prediction space

Think of the current training predictions as a vector $\mathbf{f} = (F(x_1), \dots, F(x_n))$

$$L(\mathbf{f}) = \sum_i \ell(y_i, f_i), \quad r_{im} = - \left. \frac{\partial \ell(y_i, f)}{\partial f} \right|_{f=F_{m-1}(x_i)}.$$

An unconstrained gradient step adds a multiple of r_{im} to each prediction.

Gradient descent in prediction space

Think of the current training predictions as a vector $\mathbf{f} = (F(x_1), \dots, F(x_n))$

$$L(\mathbf{f}) = \sum_i \ell(y_i, f_i), \quad r_{im} = - \left. \frac{\partial \ell(y_i, f)}{\partial f} \right|_{f=F_{m-1}(x_i)}.$$

An unconstrained gradient step adds a multiple of r_{im} to each prediction.

To predict at a new x , fit a function to these **pseudo-residuals**:

$$h_m \approx \arg \min_{h \in \mathcal{H}} \sum_i (r_{im} - h(x_i))^2.$$

The loss supplies the targets. The base learner supplies a function of x .

The gradient boosting algorithm

1. Start from a constant: $F_0(x) = \arg \min_c \sum_i \ell(y_i, c)$.
2. For $m = 1, \dots, M$, compute negative-gradient targets:

$$r_{im} = -\partial_F \ell(y_i, F_{m-1}(x_i)).$$

3. Fit a simple regression function h_m to (x_i, r_{im}) .
4. Choose a step using the *original* loss, then update:

$$\rho_m \in \arg \min_{\rho} \sum_i \ell(y_i, F_{m-1}(x_i) + \rho h_m(x_i)),$$

$$F_m = F_{m-1} + \nu \rho_m h_m, \quad 0 < \nu \leq 1.$$

Each round changes the fitting targets. Shrinkage controls how much we add.

Classification still uses the logistic loss

Return to $y \in \{0, 1\}$; $F(x)$ is a score and $\pi(x) = s(F(x))$

$$\ell(y, F) = \log(1 + e^F) - yF.$$

Classification still uses the logistic loss

Return to $y \in \{0, 1\}$; $F(x)$ is a score and $\pi(x) = s(F(x))$

$$\ell(y, F) = \log(1 + e^F) - yF.$$

$$\frac{\partial \ell}{\partial F} = s(F) - y \quad \Longrightarrow \quad r_{im} = y_i - \pi_{m-1}(x_i).$$

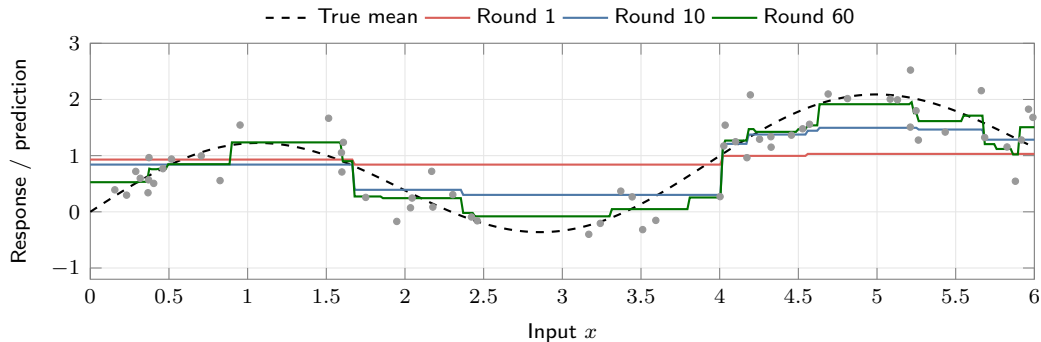
$$y = 1 : \quad \pi = 0.2 \Rightarrow r = +0.8; \quad \pi = 0.9 \Rightarrow r = +0.1.$$

$$\underbrace{x^\top \beta}_{\text{linear score}} \quad \longrightarrow \quad F_0 + \underbrace{\sum_{m=1}^M \nu \rho_m h_m(x)}_{\text{additive ensemble score}}.$$

Keep the Bernoulli likelihood. Learn a more flexible score function.

Simple corrections build a flexible fit

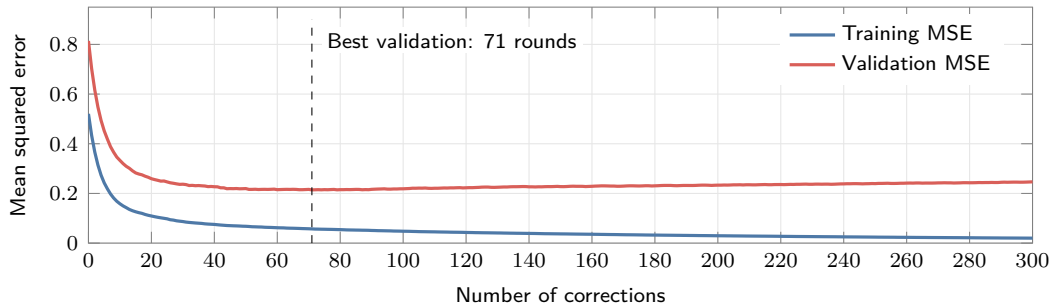
60 constructed observations; shrinkage $\nu = 0.12$; fixed data across rounds



Each correction assigns constants to at most four input intervals.

Use validation to choose when to stop

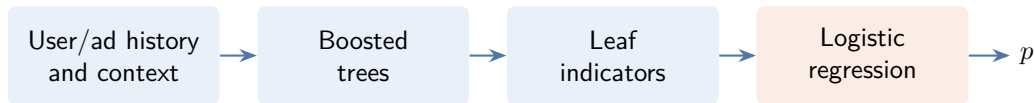
Same run: 60 training observations and 500 independent validation observations



Lower training error does not guarantee lower error on new data.

Ad clicks: learn features with trees

Facebook case study, 2014: will an ad impression receive a click?

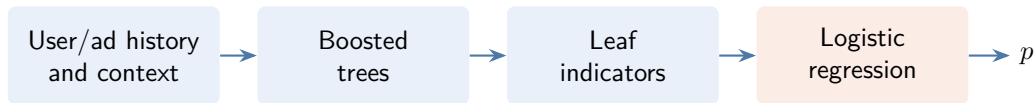


Schematic: each leaf is a region defined by a sequence of split rules.

He et al., ADKDD 2014, Fig. 1 and Sec. 3.1. [Paper](#)

Ad clicks: learn features with trees

Facebook case study, 2014: will an ad impression receive a click?



Schematic: each leaf is a region defined by a sequence of split rules.

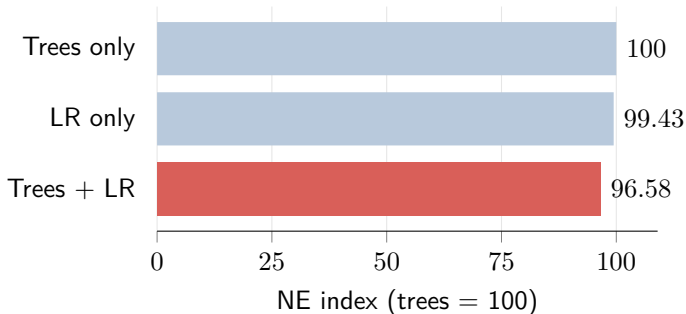
$$\phi_{m\ell}(x) = \mathbf{1}\{x \text{ reaches leaf } \ell \text{ of tree } m\}$$
$$\Pr(Y = 1 \mid x) = s\left(b + \sum_{m,\ell} w_{m\ell} \phi_{m\ell}(x)\right).$$

Trees discover interactions; logistic regression weights their rules.

He et al., ADKDD 2014, Fig. 1 and Sec. 3.1. [Paper](#)

Does the hybrid predict clicks better?

Offline comparison: Facebook ad data from one week in Q4 2013



Lower is better

$$NE = \frac{\text{mean log loss}}{H(\bar{p})}.$$

$\bar{p}$: training click rate.

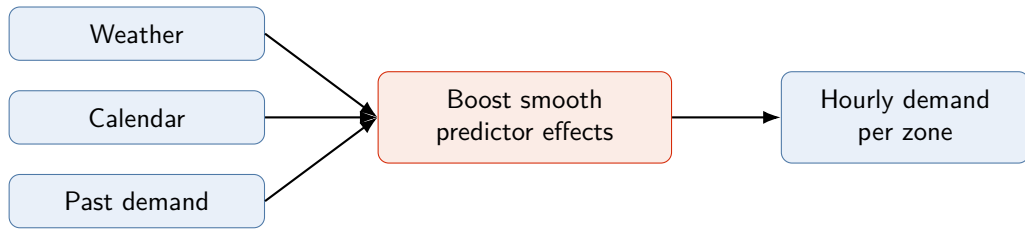
3.42% lower NE
than trees alone.

Prediction loss; no revenue or click-lift measurement.

He et al., ADKDD 2014, Sec. 2 and Table 1. [Paper](#)

Forecast electricity demand

GEFCom2012: hourly demand from a US utility



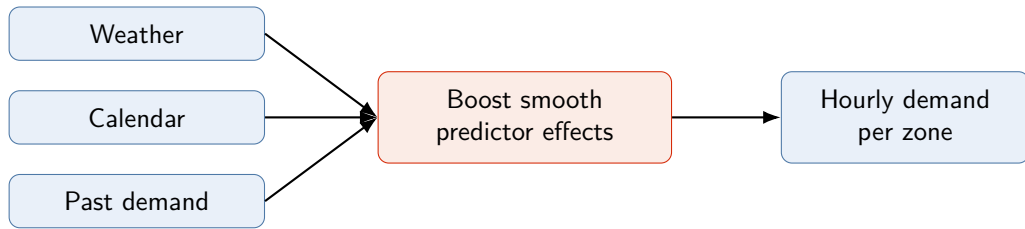
Explanatory pipeline

20 zones; 11 weather stations; 4.5 years of records.

Ben Taieb & Hyndman (2014), Abstract; Sections 3–4.

Forecast electricity demand

GEFCom2012: hourly demand from a US utility



Explanatory pipeline

20 zones; 11 weather stations; 4.5 years of records.

Team TinTin: boost **smooth spline functions**.

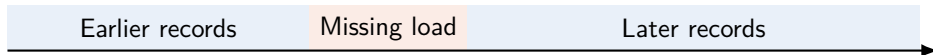
5th of 105 teams
Reported competition ranking.

Ben Taieb & Hyndman (2014), Abstract; Sections 3–4.

What is known when we predict?

Eight historical weeks to reconstruct; one future week to forecast

Historical gap



Temperature and records on both sides are available.

Future week



At the forecast cutoff, future temperature must also be predicted.

A deployment evaluation must reproduce the information available when a forecast is issued.

Ben Taieb & Hyndman (2014), Section 3. Timeline is schematic.

Phase 4: A Brief Look at Trees

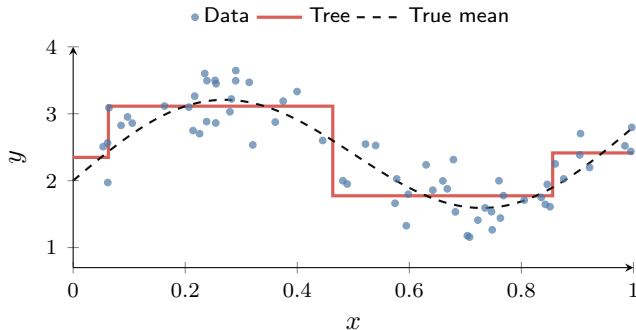
Our boosting models learned a sequence of simple corrections.

How does a decision tree
build one prediction?

Next: Partition the feature space, fit a value in each leaf, and examine what changes when the data change.

A tree predicts within regions

A brief tree recap: a depth-two tree partitions the input into four intervals.



For leaves $R_1, \dots, R_J$,

$$f(x) = \sum_{j=1}^J c_j \mathbf{1}\{x \in R_j\}.$$

Squared-loss prediction:

$$c_j = \frac{\sum_{i: x_i \in R_j} y_i}{|\{i : x_i \in R_j\}|}.$$

First split: $x \leq 0.463$.

Repeat a split within each child.

A **stump** has just one split.

Choose a split by reducing error

At a node, try a feature j and a threshold t .

$$L(j, t) = \{i : x_{ij} \leq t\}, \quad R(j, t) = \{i : x_{ij} > t\}.$$
$$(j^*, t^*) = \arg \min_{j, t} \left[\sum_{i \in L(j, t)} (y_i - \bar{y}_L)^2 + \sum_{i \in R(j, t)} (y_i - \bar{y}_R)^2 \right].$$

Six observations:

x	1	2	3	4	5	6
y	1	1	2	5	6	6

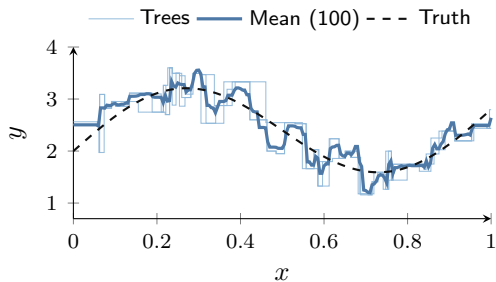
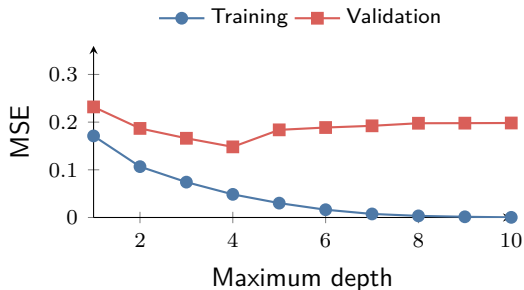
Threshold	Child means	Total SSE
2.5	1, 4.75	10.75
3.5	1.33, 5.67	1.33

Choose $t = 3.5$ here. Grow the tree **greedily**, one node at a time.

Classification: use class fractions p_k and Gini impurity $G = 1 - \sum_k p_k^2$.

Deep trees can be unstable

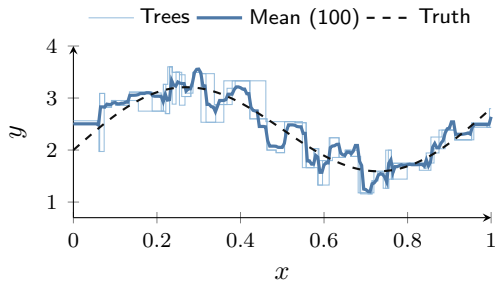
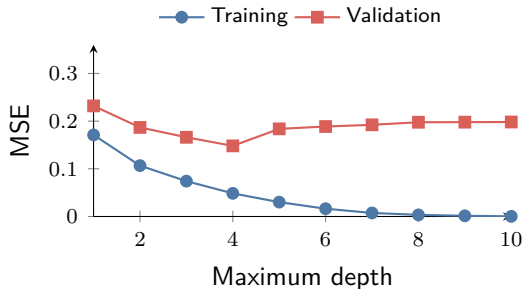
Constructed regression data: 64 training rows and independent validation data.



More depth can fit noise. Resampling the rows changes the fitted tree.

Deep trees can be unstable

Constructed regression data: 64 training rows and independent validation data.



More depth can fit noise. Resampling the rows changes the fitted tree.

Averaging 100 trees lowers validation MSE here: 0.153, compared with 0.198 for one tree.

Phase 5: Bagging

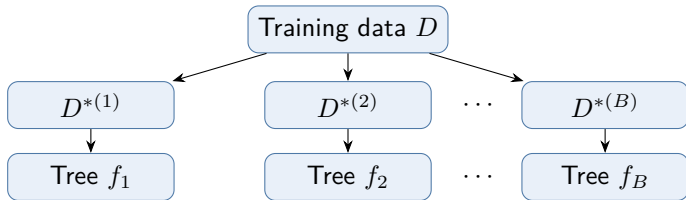
A small change in the training data can change a deep tree.

Can several unstable trees
produce a steadier prediction?

Next: Resample the observations. Fit trees independently. Average their predictions and analyze the variance.

Bagging averages bootstrap fits

Bootstrap aggregation: train trees independently on resampled data, then combine them.



Each bootstrap: n draws **with replacement**.

Fit the trees independently, then aggregate.

Regression: $\bar{f}(x) = \frac{1}{B} \sum_{b=1}^B f_b(x)$. Classification: plurality vote among tree labels.

Why averaging can reduce variance

Fix an input x . View tree predictions $f_b(x)$ as random variables.

Assume $\text{Var}(f_b(x)) = v$ and $\text{Cov}(f_b(x), f_c(x)) = \rho v$ for $b \neq c$.

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B f_b(x)\right) = \frac{1}{B^2} \left[\sum_b \text{Var}(f_b(x)) + \sum_{b \neq c} \text{Cov}(f_b(x), f_c(x)) \right]$$

Why averaging can reduce variance

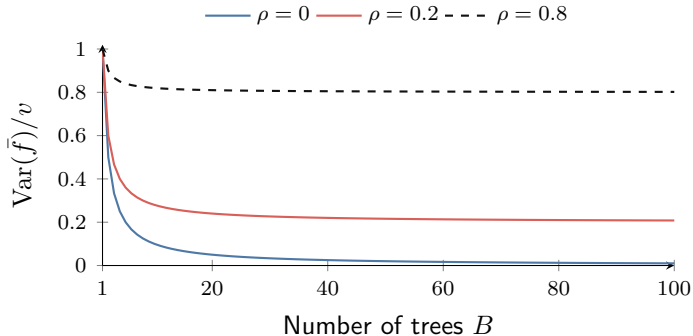
Fix an input x . View tree predictions $f_b(x)$ as random variables.

Assume $\text{Var}(f_b(x)) = v$ and $\text{Cov}(f_b(x), f_c(x)) = \rho v$ for $b \neq c$.

$$\begin{aligned}\text{Var}\left(\frac{1}{B} \sum_{b=1}^B f_b(x)\right) &= \frac{1}{B^2} \left[\sum_b \text{Var}(f_b(x)) + \sum_{b \neq c} \text{Cov}(f_b(x), f_c(x)) \right] \\ &= \frac{Bv + B(B-1)\rho v}{B^2} \\ &= \boxed{\rho v + \frac{(1-\rho)v}{B}}.\end{aligned}$$

Lower correlation makes averaging more effective.

Correlation limits the benefit



At $B = 100$:

ρ	$\text{Var}(\bar{f})/v$
0	0.01
0.2	0.208
0.8	0.802

More trees reduce the second term.

How can we also reduce correlation?

Phase 6: Random Forests

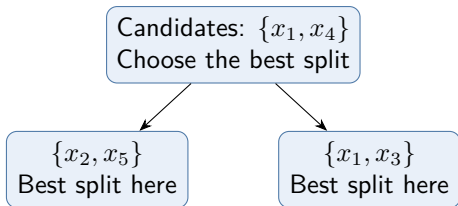
Averaging helps less when tree predictions move together.

How can we make the trees
less alike?

Next: Randomize the candidate features at every split.
Evaluate on observations omitted from each tree's training sample.

Random forests vary each split

Bootstrap the rows; sample a fresh subset of candidate features at every node.



Different nodes, different subsets.
Repeat for each bootstrap tree.

At each node:

1. Sample m of the d features.
2. Search splits using these features.
3. Keep the best split.

Smaller m can reduce tree correlation, but may weaken each tree.
Tune feature count and leaf complexity.

Out-of-bag prediction

For row i , predict using only trees whose bootstrap sample omitted row i .

A row is absent from one bootstrap with probability

$$\Pr(i \text{ omitted}) = \left(1 - \frac{1}{n}\right)^n \\ \rightarrow e^{-1} \approx 0.368.$$

Evaluate each row using its own out-of-bag trees, then average the losses.

Example: $n = 5$ rows, three bootstrap samples.

Tree	Sampled row IDs	Omitted IDs
1	1, 1, 2, 4, 5	3
2	2, 3, 3, 4, 4	1, 5
3	1, 2, 2, 3, 5	4

Row 3 is evaluated with tree 1 only in this tiny example.

Two ways to build a stronger predictor

Boosting

Fit the next learner to improve the current ensemble.

$$F_0 \longrightarrow F_1 \longrightarrow \cdots \longrightarrow F_M$$
$$F_m = F_{m-1} + \alpha_m h_m.$$

AdaBoost: reweight difficult examples.

Gradient boosting: fit negative gradients.

Often use small, restricted base learners.

Bagging / random forests

Fit diverse trees independently; average or vote.

$$D \longrightarrow \begin{cases} f_1 \\ f_2 \\ \vdots \\ f_B \end{cases} \longrightarrow \bar{f}$$

Stabilize variable predictions.

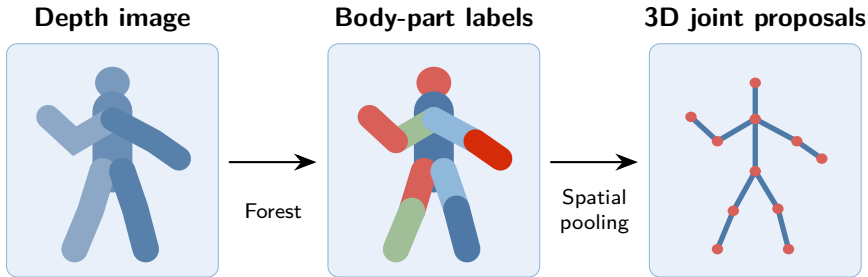
Often use deep, individually flexible trees.

Forests add random feature subsets.

Both combine models. Their training mechanisms and guarantees differ.

Kinect: trees recognize body parts

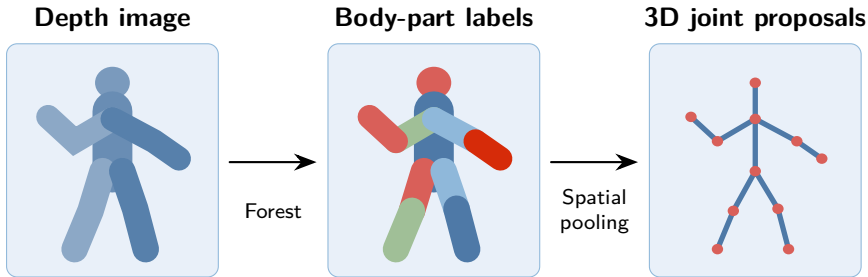
Real deployment: body-pose estimation for controller-free games.



Explanatory schematic; colors illustrate body parts.

Kinect: trees recognize body parts

Real deployment: body-pose estimation for controller-free games.



Explanatory schematic; colors illustrate body parts.

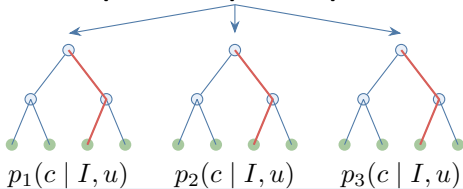
Turn a pose-estimation problem into many **body-part classification** problems.

Shotton et al., CVPR 2011, Sections 1 and 3.

From tree probabilities to joint positions

A randomized decision forest classifies each depth-image pixel.

One pixel + depth comparisons



$$p(c | I, u) = \frac{1}{3} \sum_{b=1}^3 p_b(c | I, u)$$

Explanatory trees; leaf distributions are averaged.

Published configuration

31 body-part labels

3 trees; depth 20

Reported algorithm speed

Under 5 ms/frame

Xbox 360 GPU

Camera input: 30 frames/s

Crossed arms and unseen poses can still cause errors.

Shotton et al. (2011), Sections 1, 2.1, 3.3–3.4, and 4.

How do the models work together?

Method	How the next model is fitted	How predictions combine
AdaBoost	Fit a classifier under updated observation weights.	Weighted vote through $\sum_m \alpha_m h_m(x)$.
Gradient boosting	Fit a learner to a loss-gradient target.	Add corrections to scores or responses.
Bagging / RF	Resampled observations; RF also samples candidate features.	Average values or aggregate class predictions.

The loss, the base learner, and the way models are combined define the method.

References and further reading

- Freund and Schapire (1997). *A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting*.
- Friedman (2001). *Greedy Function Approximation: A Gradient Boosting Machine*.
- Breiman (1996). *Bagging Predictors*.
- Breiman (2001). *Random Forests*.

Sources for the application cases

- Amatriain and Basilico (2012). *Netflix Recommendations: Beyond the 5 Stars (Part 1)*. Netflix Technology Blog.
- He et al. (2014). *Practical Lessons from Predicting Clicks on Ads at Facebook*.
- Ben Taieb and Hyndman (2014). *A Gradient Boosting Approach to the Kaggle Load Forecasting Competition*.
- Shotton et al. (2011). *Real-Time Human Pose Recognition in Parts from Single Depth Images*.